

Distributed Financial Transaction Processor

Final Report

Sampath Pranay Beela

Rahul China Jagadeesha

CS6650 — Building Scalable Distributed Systems

1 Problem, Team, and Overview of Experiments

This project builds a distributed financial transaction processor that accepts transfer requests at an API layer, places work on a queue, and applies account balance updates asynchronously. The core requirement is correctness under concurrency. A system that is fast but occasionally loses money, applies a transfer twice, or produces inconsistent balances is unusable for financial stakeholders. The intended stakeholders are engineers building payment or wallet infrastructure, operators responsible for system reliability, and end users who expect transfers to be durable and correct even during retries, failures, and bursty demand.

Sampath Pranay Beela focuses primarily on API design, service integration, load generation, and result analysis. Rahul China Jagadeesha focuses primarily on DynamoDB schema design, worker-side concurrency control, correctness verification, and infrastructure automation. The project is intentionally split this way so that request ingestion, asynchronous execution, and correctness checks can evolve in parallel while still meeting at clear integration boundaries.

The architecture uses a Go-based `api-svc`, a Go-based `worker-svc`, an SQS-compatible queue, and DynamoDB as the system of record. The current implementation records transfers as `PENDING`, enqueues them, and completes them in the worker using conditional writes on account versions. Observability is treated as a first-class concern. The current local stack exposes a health endpoint, transaction status records, service logs, Locust HTML/CSV exports, and a balance reconciliation script that checks the total system balance after each run. The planned cloud deployment extends this with CloudWatch metrics and logs for ECS tasks, queue depth, and DynamoDB consumption.

The experimental plan stresses the system in four ways. Experiment 1 is a hot-account contention storm in which many users transfer from the same sender account. Experiment 2 kills a worker after dequeue but before commit to study retry safety and idempotency. Experiment 3 compares optimistic locking against a more coordinated pessimistic alternative. Experiment 4 scales API and worker replicas from one instance to several instances while re-running correctness checks. Across these experiments, the measured outputs are throughput, p95 latency, retry and conflict behavior, failure rate, and final balance correctness.

AI is used as a development accelerant rather than as an authority. It is useful for scaffolding boilerplate, drafting Terraform and LaTeX, and generating first-pass load-test code, but it is not trusted to validate concurrency correctness or interpret pathological behavior. Human review remains necessary for experimental design, debugging race conditions, and judging whether measured tradeoffs are acceptable.

This framing is important because the project is not merely building another CRUD backend. A transfer processor is a compact way to study several hard distributed-systems problems at once: asynchronous execution, duplicate delivery, write contention, rollback difficulty, and the gap between request success and durable correctness. In a read-mostly service, a transient inconsistency may be acceptable. In a financial service, even a rare inconsistency can destroy trust. The report therefore emphasizes not only measured performance but also the evidence used to argue that the measured performance corresponds to a correct system.

2 Project Plan and Recent Progress

The recent milestone focused on establishing a working local prototype before moving to cloud deployment. The repository now contains a shared Go module, both services, Dockerfiles, Docker Compose orchestration, queue and DynamoDB wiring, a seed script for 101 accounts, Locust load tests, and a balance verification script. The local workflow is automated through `make init`, `make smoke`, `make test-load`, `make test-load-hot`, and `make verify`. The progress log reports that the system builds cleanly, the smoke test completes end-to-end, and both load tests finish with zero failures while preserving global balance correctness.

The detailed project plan is summarized in Table 1. The sequence is deliberate: local correctness first, then repeatable infrastructure, then comparative experiments, and finally synthesis. This ordering reduces the risk of spending time on cloud deployment before the core transfer semantics are stable.

AI affects this plan in both positive and negative ways. The main benefits are faster generation of service skeletons, test harnesses, and documentation drafts. The main costs are verification time and the risk of subtle hallucinations

Table 1: Project plan and ownership

Phase	Key tasks	Owner(s)	Timeline	Success criterion
Local core	Service scaffolding, DynamoDB schema, idempotency, optimistic locking, queue wiring	Both	Weeks 1–2	End-to-end transfer flow works locally and balances reconcile
Containerization and infra	Docker images, Compose, Terraform, and cloud resource setup	Rahul, Pranay	Weeks 3–4	System deploys reproducibly and passes smoke tests
Preliminary experiments	Mixed-load test, hot-account test, balance verification, metrics export	Pranay	Weeks 3–4	Stable baseline metrics captured with zero correctness violations
Comparative concurrency study	Pessimistic locking implementation and side-by-side comparison	Both	Weeks 5–6	Breakpoint between strategies identified under increasing load
Scaling study	Increase replica count and re-run correctness checks	Both	Weeks 5–6	Throughput scaling measured without breaking invariants
Final analysis	Charts, report writing, demo preparation	Both	Weeks 7–8	Final narrative and evidence are ready for submission

in cloud configuration, retry logic, or concurrency reasoning. For this project, the cost-benefit balance is favorable only when AI output is constrained by measured results, code review, and deterministic validation such as balance reconciliation.

Recent progress is substantial enough to de-risk the rest of the schedule. The team has already replaced the earlier LocalStack assumption with purpose-built local services for DynamoDB and SQS, pinned the shared Go dependencies, and verified that both services build and run cleanly. The seeded environment now creates 100 ordinary accounts plus one dedicated hot account, each initialized to \$10,000, for a total expected balance of \$1,010,000. That fixed baseline is critical because every subsequent experiment can be judged against the same conserved total.

The role split is also now concrete rather than aspirational. Pranay’s work has concentrated on the external system surface area: request handling, queue integration, Dockerization, and Locust-based workload generation. Rahul’s work has concentrated on internal correctness: versioned account updates, worker retry behavior, verification scripts, and infrastructure provisioning. This separation matters because it keeps the team from coupling load-generation progress to concurrency-control implementation details, while still forcing integration at the queue and transaction-table boundaries.

3 Objectives

The short-term objective is to deliver a transaction processor that enforces three invariants: transfers are idempotent, account balances remain non-negative and globally conserved, and retries do not create double spending. A second short-term objective is to quantify how well optimistic locking performs under both mixed traffic and concentrated contention. A third short-term objective is to make the system observable enough that correctness bugs are diagnosable rather than speculative.

The long-term objective extends beyond the course. The system should evolve into a reusable benchmark for studying correctness-versus-throughput tradeoffs in financial write-heavy workloads. That means adding richer failure injection, dead-letter queue analysis, request tracing, cloud-native observability, and a second coordination strategy that can be compared under identical workloads. In a production setting, stakeholders would also require authentication, authorization, auditability, and cost-aware autoscaling. If AI-assisted development remains part of the workflow, reliability and cost control must come from strict prompt scoping, code review, test gates, and a bias toward measured behavior over generated explanation.

An equally important objective is explanatory power. The project should not only report that one strategy is faster than another; it should explain why that strategy wins under a given workload shape. That means the final

system must support reasoning from symptoms to causes: a spike in p95 latency should be relatable to contention, queue behavior, retry backoff, or datastore coordination cost. Observability is therefore part of the objective set, not a side tool.

The long-term vision also includes portability. Although the current implementation is framed around a course milestone and an AWS-oriented deployment path, the underlying design questions are larger: when is an eventually retried optimistic design good enough, and when should a team pay for stronger coordination? By making those tradeoffs explicit, the project can become a reusable teaching artifact for future iterations of the course or for other students testing similar write-heavy systems.

4 Related Work

This project sits in the same design space as course topics on asynchronous messaging, at-least-once delivery, optimistic concurrency control, and the operational cost of distributed coordination. The use of a queue between request ingestion and execution follows the standard distributed-systems pattern of decoupling bursty front-end load from back-end mutation logic. The use of DynamoDB conditional writes reflects the broader class of optimistic concurrency techniques that favor throughput under low contention while pushing complexity into retries under contention.

The project also relates to a broader engineering pattern visible in production payment and approval systems: the API path should do enough work to establish intent durably, but not so much work that user-facing latency becomes coupled to every downstream state mutation. Recording the transaction and enqueueing it before the worker performs the account mutation mirrors that pattern. It also exposes the exact place where idempotency must hold, because a queue with at-least-once delivery semantics can redeliver the same request even after partial progress.

Three Piazza projects are especially close in spirit. Tejas Shinde’s distributed live leaderboard is similar because it studies fault tolerance, shard failover, and horizontal scaling under stress. The difference is that a leaderboard can often tolerate brief staleness or partial answers, while this project cannot tolerate any inconsistency in monetary state. Gideon Ntim Gyakari’s approval workflow is similar because it also uses SQS and idempotency to tolerate duplicate delivery under load. The difference is the consistency mechanism: that project uses PostgreSQL with serializable isolation, while this system uses DynamoDB conditional writes and explicit version checks. Harrison Pham’s concert ticket flash sale platform is similar because correctness under intense contention is central to both systems. The difference is that ticketing protects limited inventory, whereas a financial processor must also preserve a conserved quantity across debit and credit operations, making rollback and retry behavior more sensitive.

Together, these comparisons locate the project in an interesting middle ground. It is more correctness-sensitive than leaderboard-style aggregation, more queue-centric than a traditional relational transaction processor, and more state-conserving than a ticketing system that only prevents oversell. That makes it a useful benchmark for distributed-systems discussions because it forces attention to both latency and invariants.

5 Methodology

The implementation follows a queue-backed asynchronous model. The API service validates requests, checks whether the transaction identifier already exists, records a **PENDING** transaction with a conditional write, and then enqueues the request. The worker polls the queue, reads the sender account with a consistent read, verifies sufficient funds, performs a version-checked debit, reads and credits the receiver, and finally marks the transaction **COMPLETED**. Version conflicts trigger up to three retries with exponential backoff. Non-recoverable conditions such as missing accounts or insufficient funds mark the transaction **FAILED**.

This methodology exposes the exact tradeoff the project is meant to study. Optimistic locking keeps the steady-state fast because no distributed lock service is involved, but it shifts the burden to retry handling when many writers target the same item. The pathological workload is therefore a hot-account storm in which all users compete to debit the same source account. The base case is a mixed workload over many accounts, which should

produce lower contention and better throughput. Worker crash testing and future pessimistic-locking experiments are designed to separate correctness guarantees from performance claims.

Observability is built into the method rather than added afterward. API responses expose transfer status, worker logs show retry behavior and failures, Locust exports latency and throughput histories, and the verifier scans the accounts table to ensure that the seeded total balance of \$1,010,000 is preserved to within a one-cent tolerance. This combination gives the project three distinct lenses: request-level performance, control-plane behavior during retries, and end-state financial correctness.

The experimental method is intentionally incremental. Before any comparative claim is made, the system must first survive the smoke test and preserve balances after synthetic load. Only then does it make sense to compare contention patterns or alternative locking designs. This prevents a common distributed-systems mistake in which throughput numbers are gathered from a system whose correctness story is still weak or partially unverified.

Each planned experiment isolates a different mechanism. The hot-account test isolates version conflict pressure on a single sender. The worker-crash test isolates the interaction between queue delivery and incomplete work. The optimistic-versus-pessimistic comparison isolates coordination strategy. The horizontal-scaling test isolates whether adding replicas changes only throughput or also changes correctness behavior. By designing the experiments around isolated questions, the team can attribute observed changes to one principal cause rather than to several moving parts at once.

AI use within the methodology is narrow on purpose. It can help produce initial implementations of handlers, queue clients, load-test scripts, or documentation, but all generated artifacts are checked against code behavior and measured outputs. In this project, a generated explanation of concurrency is far less valuable than a concrete invariant check that passes after stress. The verification script and the load-test exports therefore carry more epistemic weight than any generated prose.

6 Preliminary Results

Table 2 summarizes the current measured results from the provided load-test artifacts. The mixed run combines random transfers and hot-account transfers for five minutes with 50 users. The isolated hot-account run uses 50 users hammering a single sender account for five minutes. In both runs, the failures and exceptions CSV files are empty, indicating that Locust observed no request-level failures or worker-side exception surfacing.

Table 2: Measured preliminary results from `load-tests`

Scenario	Req.	Avg ms	p95 ms	p99 ms	Avg req/s	Peak req/s
Mixed load aggregated	81,152	4.32	10	32	270.84	280.7
Mixed random transfers	24,236	4.44	11	35	80.89	—
Mixed hot transfers	56,916	4.27	10	31	189.95	—
Hot-account only	113,950	3.86	7	28	380.38	394.7

Two findings stand out. First, the system sustained 270.84 requests/s in the mixed run and 380.38 requests/s in the hot-account-only run while keeping request-level failure rate at 0%. Second, the hot-account-only scenario did not cause the dramatic latency collapse that an optimistic-locking design might fear at this scale; its p95 latency was 7 ms, better than the 10 ms aggregated p95 in the mixed run. The history files show that the hot-account scenario reached 394.7 requests/s at peak while the mixed run peaked at 280.7 requests/s. This suggests that, at 50 concurrent users and with the current retry budget, the implementation absorbs conditional-write conflicts without surfacing instability to the client.

The mixed-run breakdown is also informative. Of the 81,152 total requests in the mixed scenario, 56,916 came from the hot-transfer workload and 24,236 came from random transfers. Despite this blend of low- and high-contention paths, the average latency remained only 4.32 ms and the p99 remained 32 ms. The random-transfer class posted a slightly higher p95 of 11 ms than the hot-transfer class at 10 ms, which indicates that at this current concurrency level the system is not yet bottlenecked by conditional-write conflicts alone. Other factors such as request mix, queue scheduling, or cold-start effects in the early portion of the run may matter more than pure contention.

The hot-account-only run is the strongest preliminary result so far because it tests the exact case that should hurt optimistic concurrency the most. It completed 113,950 requests with a median latency of 2 ms, average latency of 3.86 ms, and p99 of 28 ms while still reporting zero failures. That does not prove that optimistic locking will remain superior at 200 or 500 users, but it does establish that the current implementation is stable at the 50-user stress level and deserves the deeper comparison planned for later milestones.

To make the remaining scope explicit, Table 3 separates completed evidence from future analysis.

Table 3: Status of experiments

Experiment	Status		Current evidence	Remaining work
Mixed baseline load	Completed locally	lo-	81,152 requests, 0 failures, 270.84 req/s aggregated	Re-run after cloud deployment for comparison
Hot-account contention	Completed locally	lo-	113,950 requests, 0 failures, p95 7 ms, peak 394.7 req/s	Increase concurrency toward proposed 500-user level
Worker crash recovery	Planned		Design and rationale defined in proposal/update	Inject failures after dequeue and measure re-delivery outcomes
Optimistic vs. pessimistic locking	Planned		Optimistic baseline implemented and measured	Implement coordinated variant and compare breakpoints
Horizontal scaling verification	Planned		Local single-environment baseline established	Run on ECS at 1, 2, and 4 replicas with correctness checks

The current results are still preliminary because only the local mixed-load and hot-account tests have been completed. The remaining work is to run explicit crash-recovery experiments, implement the pessimistic-locking baseline, measure where retry overhead becomes unacceptable as concurrency rises toward the proposed 500-user stress level, and repeat the analysis under horizontal scaling in ECS. The worst-case workload remains concentrated writes to a single sender account because every debit competes on the same versioned item. The base-case workload is the random multi-account transfer stream because conflicts are naturally spread across many keys.

7 Impact

This project matters because it turns a common distributed-systems claim, “eventual correctness under retries,” into something measurable for a write-heavy financial workload. Engineers can use the resulting system and experiments to reason about when conditional writes are sufficient, when stronger coordination is worth the overhead, and how much observability is needed before trusting a queue-backed money movement pipeline. The work is also reusable within the class because other teams can adapt the load harness, verification approach, or failure-injection ideas to their own systems.

The broader significance is practical. Many distributed applications can hide small inconsistencies behind caching or reconciliation, but a financial transaction processor cannot. A clear experimental comparison among optimistic locking, failure recovery, and horizontal scaling produces evidence that is useful beyond the course: it informs how one would design payment services, wallet backends, or approval systems where correctness violations have direct business cost. That is why the emphasis in this report is not only on throughput, but on proving that the measured performance is attached to a system that still preserves its invariants.